Convolutional Neural Network

미래연구소 16기 8주차

0. CNN의 활용 분야 1) image

1> Image classification

Classification

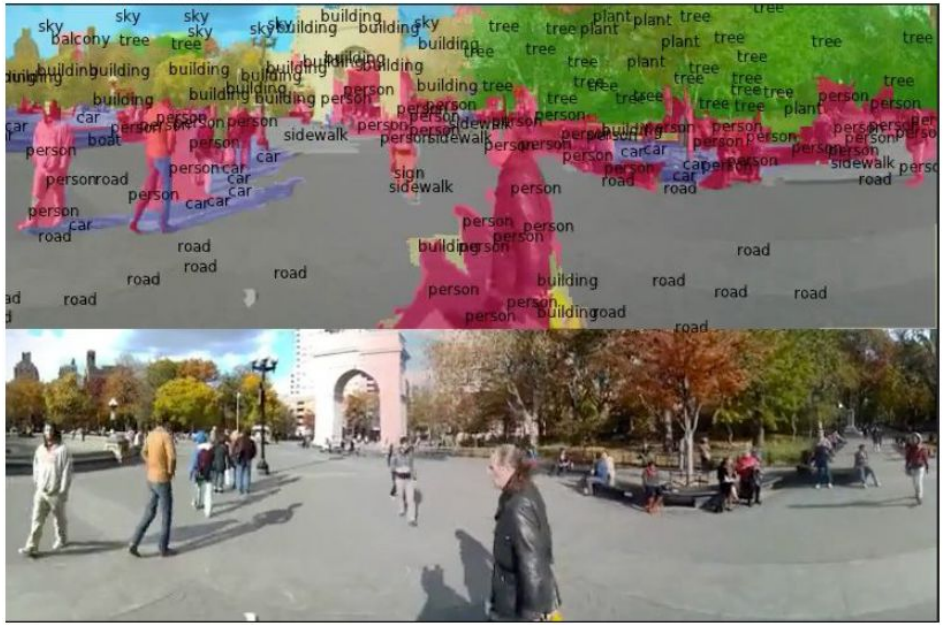
0. CNN의 활용 분야 1) image

2> object detection

Detection

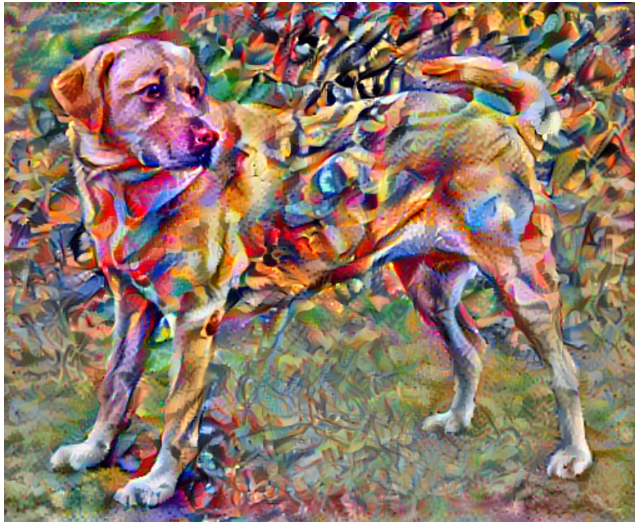
3> segmentation

Segmentation

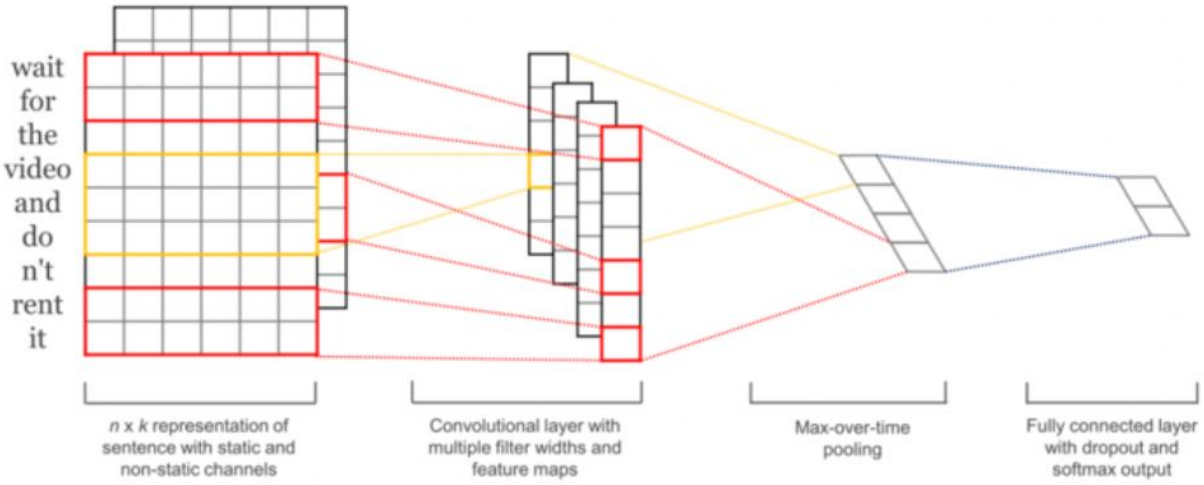


0. CNN의 활용 분야 1) image

4> Neural Styler Transfer



0. CNN의 활용 분야 2) NLP



Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification

GPU 연산에 최적화 되어 있는 구조라 연산이 빠릅니다.
NLP 뿐 아니라 길이가 긴 sequence를 처리하는데도 RNN 대신 사용한다.

0. CNN의 활용 분야 3) 그 외

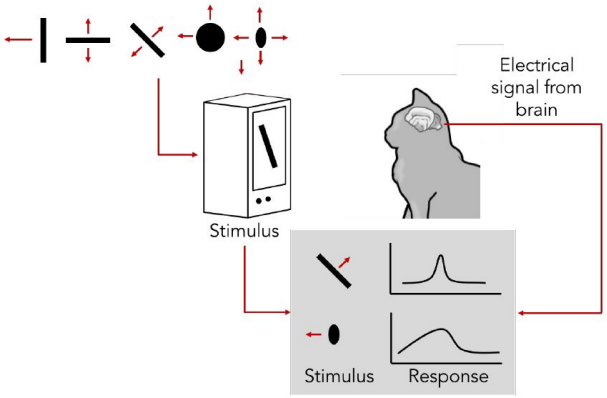
1> 알파고의 내부구조도 CNN으로 구성되어 있습니다.

2> ‘image => 언어로 설명’

등등 본인의 분야와 상관없이 CNN을 배울 필요가 있습니다.

1. 사람(동물)의 시각자극 처리 방법

1) 뇌는 시각적으로 작은 자극들부터 반응한다.



Cat image by CNX OpenStax is licensed under CC BY 4.0; changes made

1> 간단한 점, 선들부터 뉴런이 반응한다.

2> 선(물체)의 모양, 움직임마다 반응하는 뉴런들이 다르다.

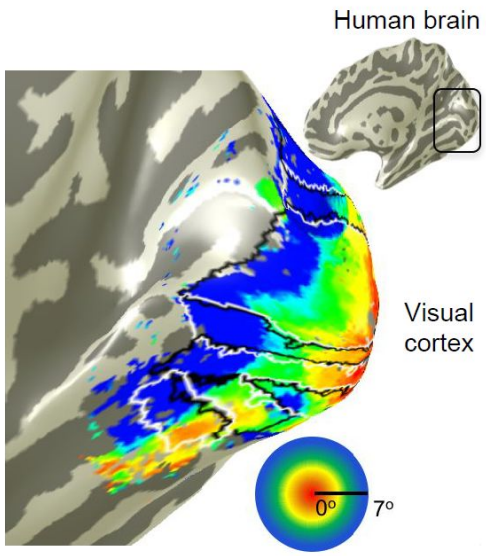
결론1: 특정 feature를 detect하는 뉴런이 필요하다.

1. 사람(동물)의 시각자극 처리 방법

2) 뇌는 인접 정보를 읽는다.

Topographical mapping in the cortex:
nearby cells in cortex represent
nearby regions in the visual field

1> 시각피질에서 가까운 신경 세포가
가까운 지역의 **feature**를 추출



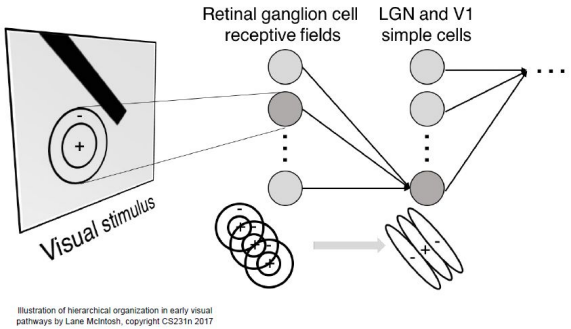
Retinotopy images courtesy of Jesse Gomez in the Stanford Vision & Perception Neuroscience Lab.

결론2: 인접 feature를 detect하는 뉴런이 필요하다.

1. 사람(동물)의 시각자극 처리 방법

3) 뇌는 공간 정보를 계층적으로 학습한다.

Hierarchical organization



Simple cells:
Response to light orientation

Complex cells:
Response to light orientation and movement

Hypercomplex cells:
response to movement with an end point

No response

Response (end point)

1> 앞 쪽의 뉴런에서는
빛 또는 물체의 존재, 방향에 **activation**

2> 그 다음 뉴런에서는
이들의 움직임에 **activation**

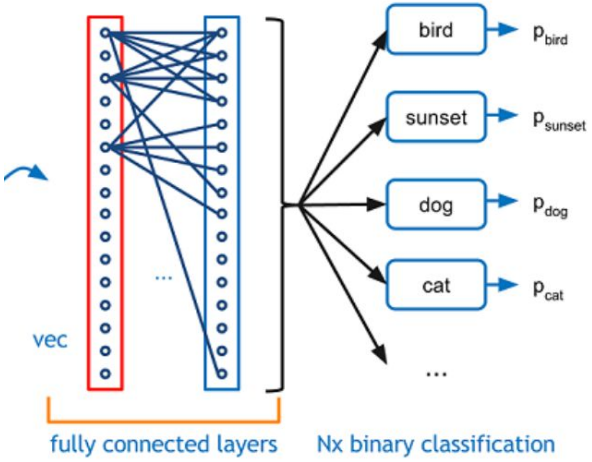
3> 더 깊은 뉴런에서는
물체의 끝 점까지도 파악 가능

결론3: feature를 계층적으로 학습할 필요가 있다.

2. Fully-Connected Layer 1) 문제점

3d array(image)를
1d array(feature vector)로

flatten



1> 변형(이동)되면 인식 못하는 경우
⇒ 물체가 이동해도 변하지 않는 특징을 읽을 필요

2> 이미지의 모든 pixel간의 feature 추출
⇒ 비효율적

결론 : 1> 필요한 정보만 2> 조금씩 읽자

3. Convolutional Layer 1) convolution 연산

실제 신호처리에서 사용되는 **convolution**과 일부 다릅니다.
convolution의 의미적인 부분만 가져왔습니다.

3. Convolutional Layer 1) convolution 연산

0	0	1	0	0
0	1	0	1	0
1	0	0	0	1
0	1	0	1	0
0	0	1	0	0

Input data

0	0	1
0	1	0
1	0	0

Filter
= Kernel

3. Convolutional Layer 1) convolution 연산

element wise product

0 0	0 0	1 1	0	0
0 0	1 1	0 0	1	0
1 1	0 0	0 0	0	1
0	1	0	1	0
0	0	1	0	0

Input data

0	0	1
0	1	0
1	0	0

Filter
= Kernel

3

3. Convolutional Layer 1) convolution 연산

0 0	0 0	1 0	0 1	0
0 0	1 0	0 0	1 0	0
1 1	0 0	0 0	0 0	1
0	1	0	1	0
0	0	1	0	0

Input data와 Filter의 convolution 연산

element wise product

0	0	1
0	1	0
1	0	0

3	0

남은 7가지 경우도 연산해보세요.

3. Convolutional Layer 1) convolution 연산

0	0	1	0	0
0	1	0	1	0
1	0	0	0	1
0	1	0	1	0
0	0	1	0	0

Input data

이게 무슨 의미가 있을까요?

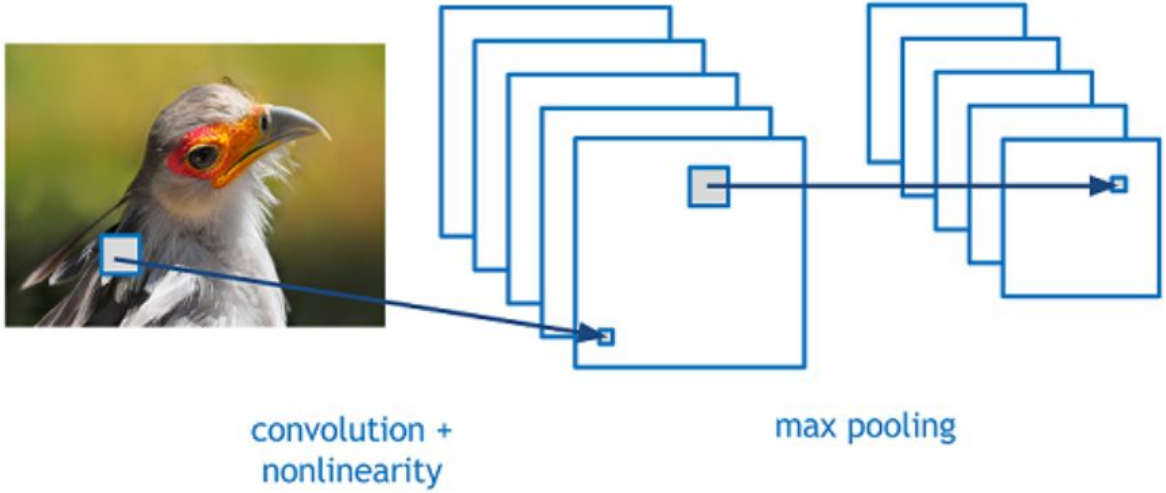
0	0	1
0	1	0
1	0	0

Filter
= Kernel

3	0	1
0	2	0
1	0	3

Activation map
= Output

3. Convolutional Layer 2) 장점: locally-connected



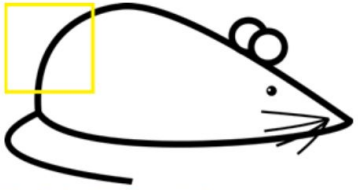
현재 **pixel**의 가로세로 크기 만큼의 주변 **pixel**의 연관관계 계산

⇒ 인접한 정보끼리만 연산

⇒ 효율적

3. Convolutional Layer 3) 장점: convolutional filter

Original image



Visualization of the filter on the image

Remember, what we have to do is multiply the values in the filter with the original pixel values of the image.

Visualization of the receptive field

0	0	0	0	0	0	30
0	0	0	0	50	50	50
0	0	0	20	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0

Pixel representation of the receptive field

*

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter

input의 위치와 관계없이, filter를 sliding window 하고 element wise product

⇒ 똑같은 **feature**라면 위치와 상관없이 해당 **feature** 추출 가능

⇒ 또한, **3X3**의 정보를 한 점으로 압축해서 표현

아까 **Fully-Connected Layer**의 **2**가지 문제점이 해결되었습니다.

1> 모든 **pixel**간의 **feature** 추출로 비효율적

=> locally-connected한 **feature** 추출

2> 물체가 이동하면 인식하지 못함

=> convolution filter는 움직이면서 **feature**를 추출한다.

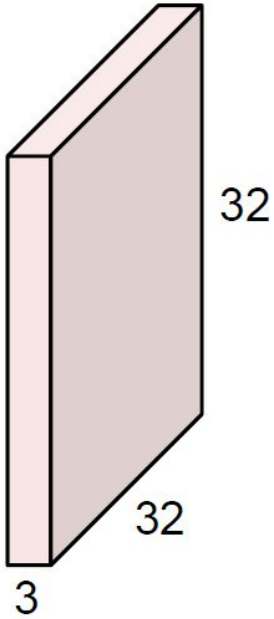
3. Convolutional Layer 4) 3D image convolution

1> C4 W1 L6 강의 (10분)

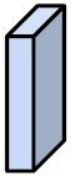
Convolution Layer

Filters always extend the full depth of the input volume

32x32x3 image

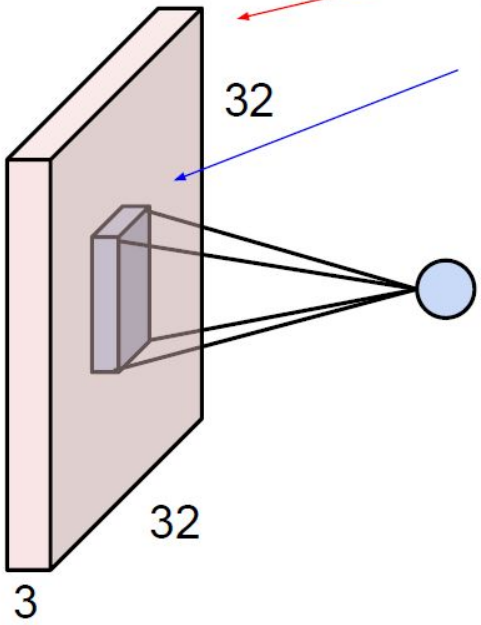


5x5x3 filter



Convolve the filter with the image i.e. “slide over the image spatially, computing dot products”

Convolution Layer



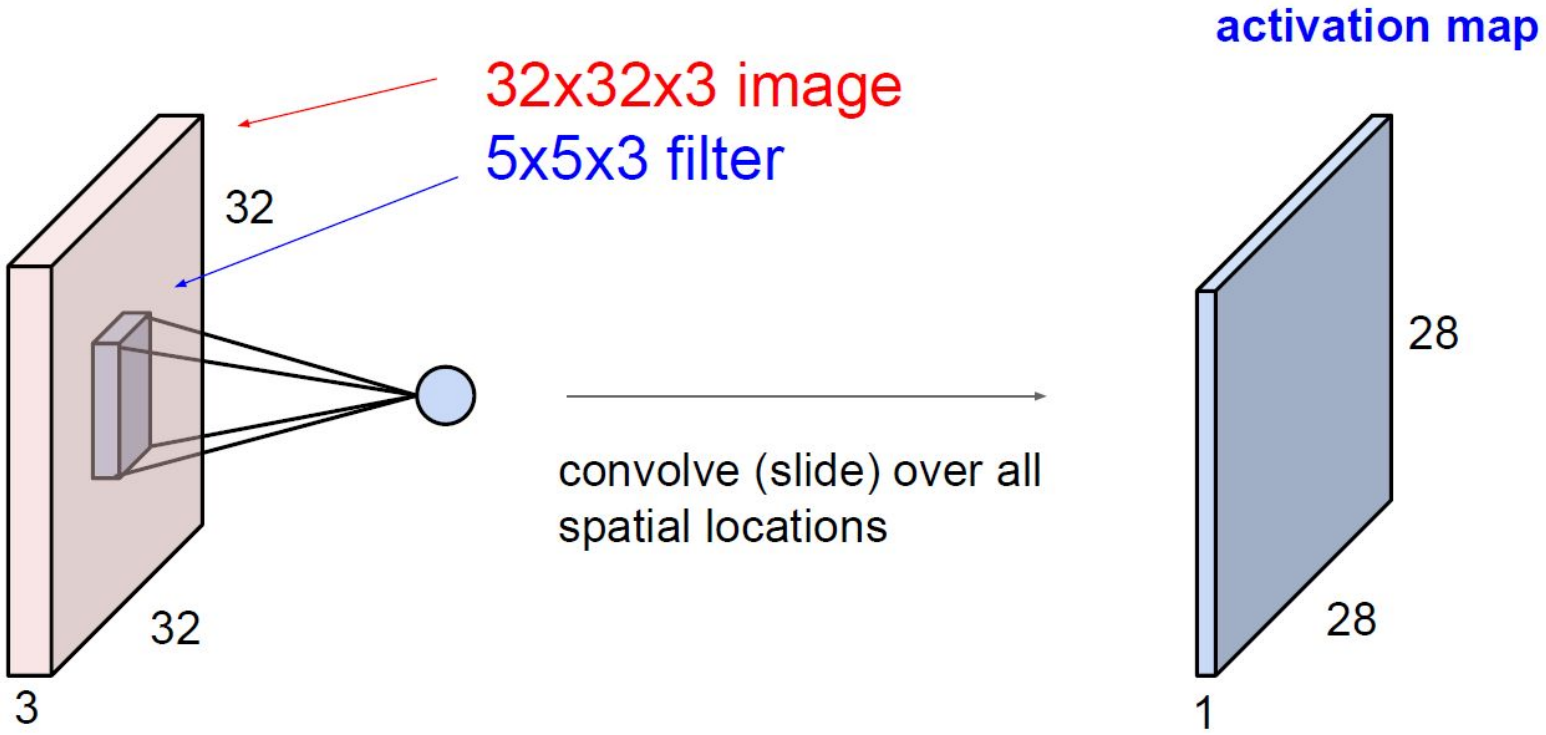
32x32x3 image

5x5x3 filter w

1 number:
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image (i.e. 5*5*3 = 75-dimensional dot product + bias)

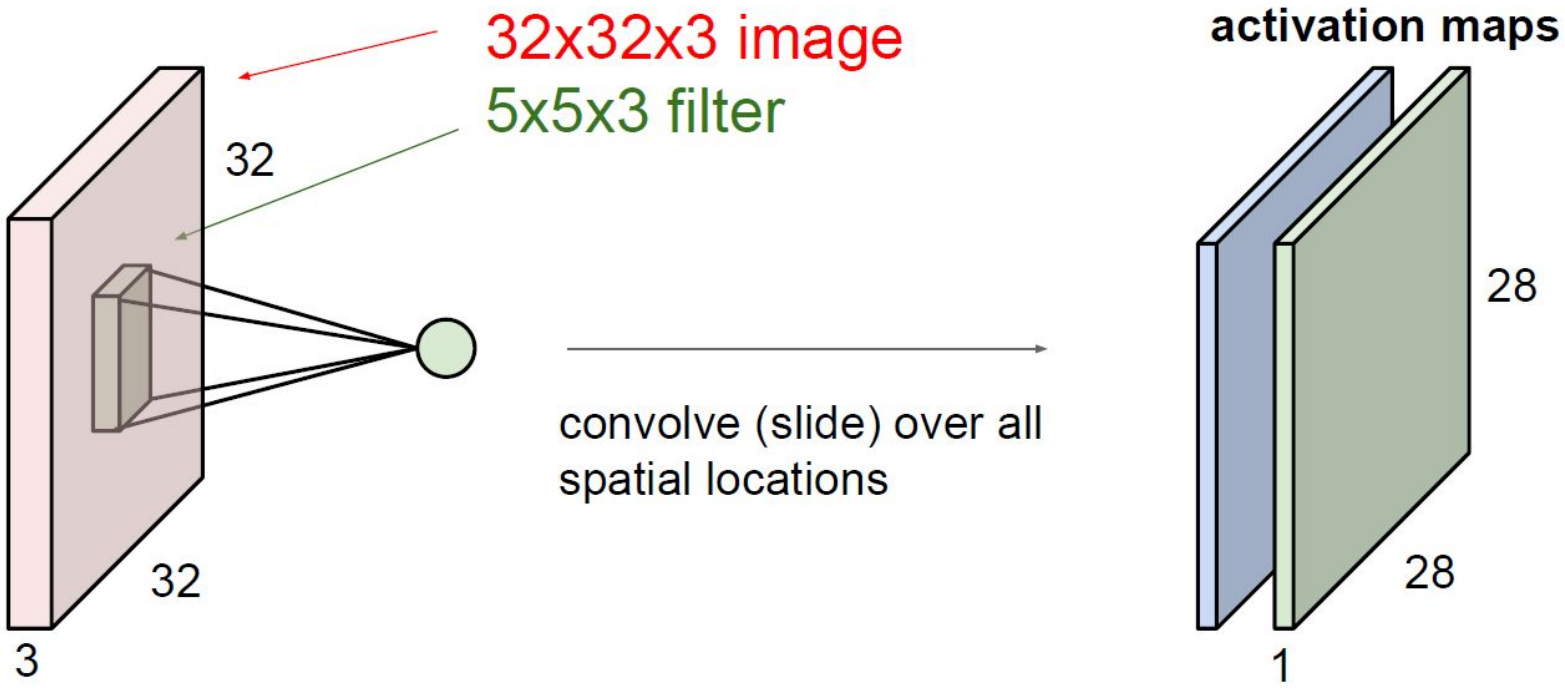
$$w^T x + b$$

Convolution Layer

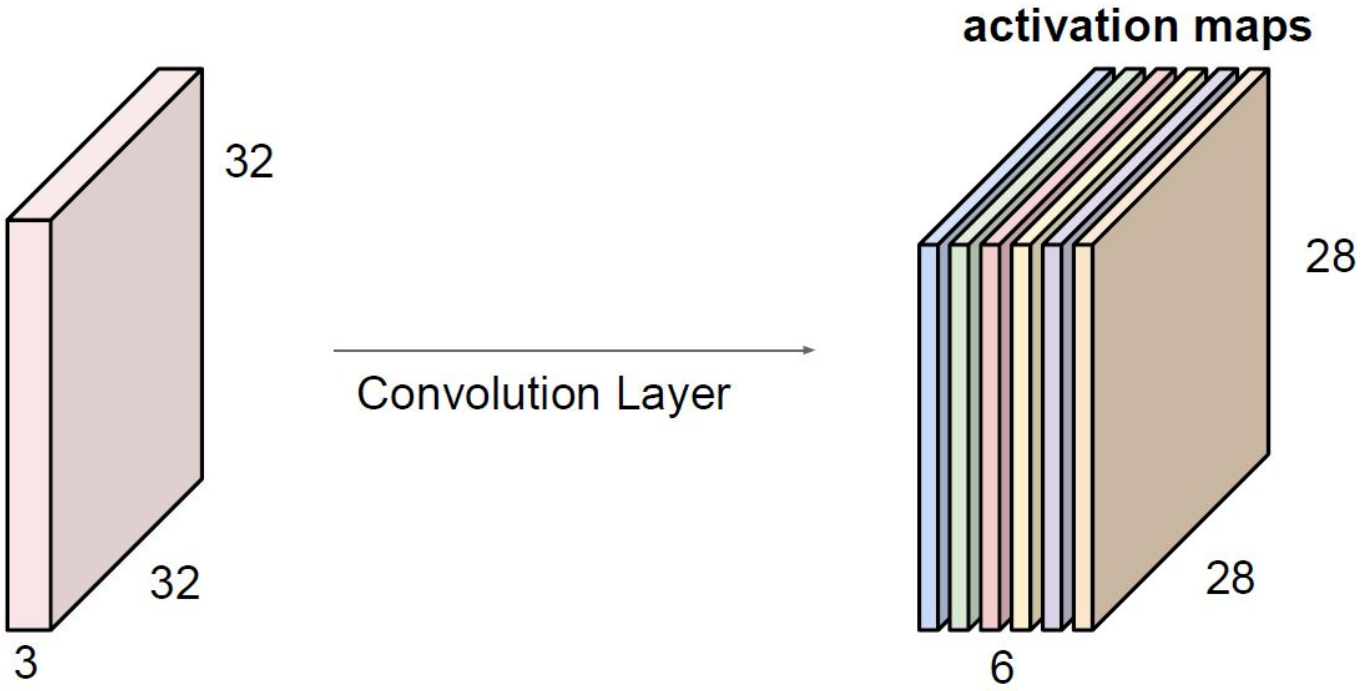


Convolution Layer

consider a second, green filter



For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



We stack these up to get a "new image" of size 28x28x6!

3. Convolutional Layer 5) 용어 정리

1> input data

2> filter = kernel = window

⇒ filter 모양의 feature 추출

⇒ filter 개수가 많을수록 더 다양한 feature 추출

3> activation map = feature map

⇒ filter가 찾은 특성이 얼마나 있는지 압축해서 표현(지도처럼 압축해놓았다.)

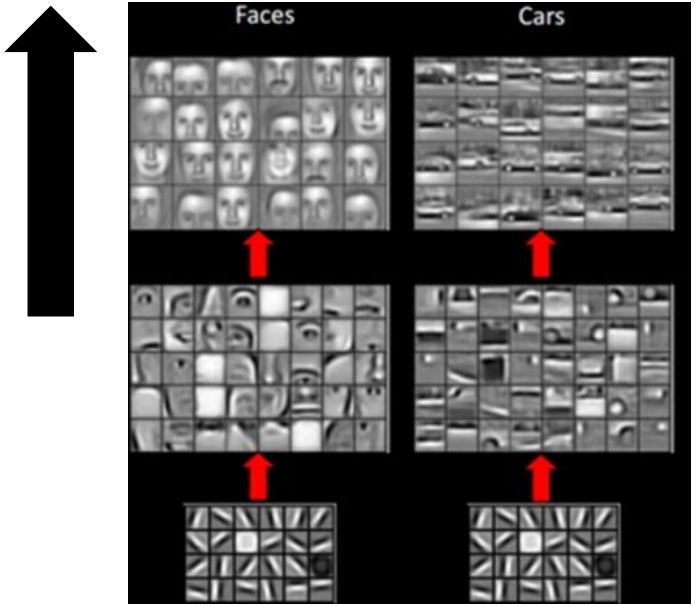
⇒ filter 개수 = output activation map의 depth

filter가 의미있는 feature를 학습할 수 있게
loss를 설정해서 loss를 줄이는 방향으로 학습

filter(Conv) = Weight(Dense)

3. Convolutional Layer 6) deep CNN

deep

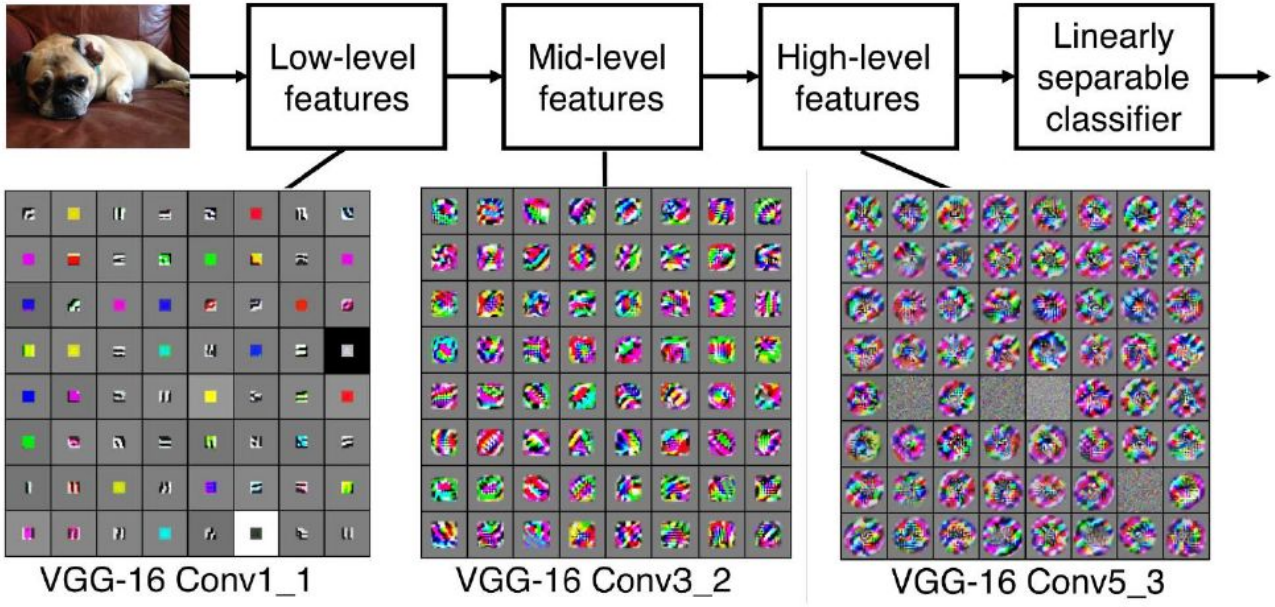


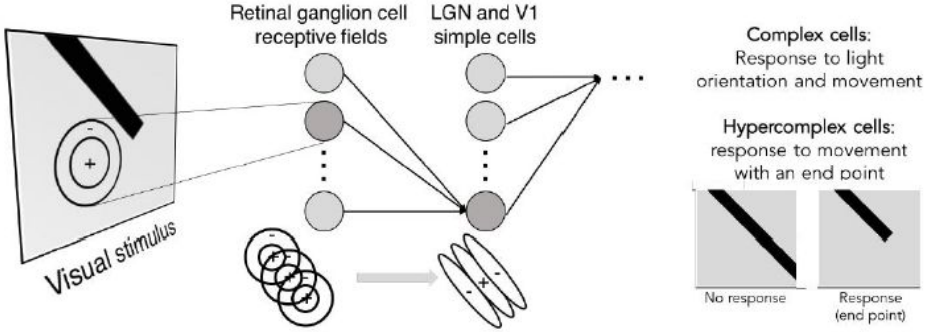
1> 앞의 layer는 단순한 edge feature 추출

2> 다음 layer는 이들을 convolution해서
더 복잡한 feature 추출

3> 계속해서 convolution으로 공간 정보를 압축
⇒ 깊을수록 크고 추상적인 feature 추출 가능

결론: 공간적인 계층 구조를 학습
(점 -> 짧은 선 -> 긴 선 -> 작은 물체 -> 큰 물체)
(인간과 유사)


Low-level features
Mid-level features
High-level features
Linearly separable classifier
VGG-16 Conv1_1
VGG-16 Conv3_2
VGG-16 Conv5_3


Retinal ganglion cell receptive fields
LGN and V1 simple cells
Visual stimulus
Complex cells:
Response to light orientation and movement
Hypercomplex cells:
response to movement with an end point
No response
Response (end point)

상세한 건 천천히 외우셔도 됩니다.

3. Convolutional Layer 6) hyperparameter

1> kernel_size

⇒ 홀수를 쓰는 것이 기본

⇒ 대부분 3X3을 사용 (5X5, 7X7)

2> filters = kernel의 개수

⇒ 자기 맘대로 (Dense layer에서 unit 개수와 유사)

⇒ 많을 수록 더 많은 feature 뽑지만 overfitting 위험

[kernel_size를 홀수를 쓰는 이유]

[1] 현재 위치 pixel에서의 가로세로 대칭만큼과의 관계를 파악하는 것이 목적

[2] 어느 pixel에서 conv 사용했는지 1:1 연결이 불가

[3] padding을 사용할 때 비대칭으로 padding을 사용하게 된다.

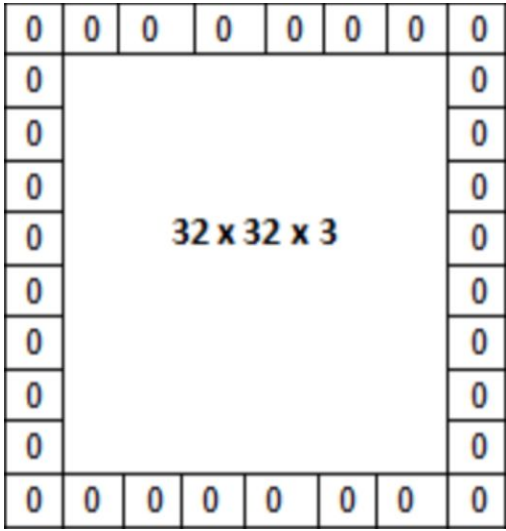
4. CNN의 다른 옵션 1) padding

1> 정의: output size가 줄어드는 것을 막기 위해
input data의 외곽을 특정 값으로 덧대준다.

2> edge 부근의 feature도 잘 추출할 수 있다.

3> 일부를 0으로 해서 원래의 feature를 희석하니 overfitting 방지하는 효과도 일부 있습니다.

(Dropout도 일부 node가 0이되면서 overfitting 막았습니다.)



4. CNN의 다른 옵션 1) padding

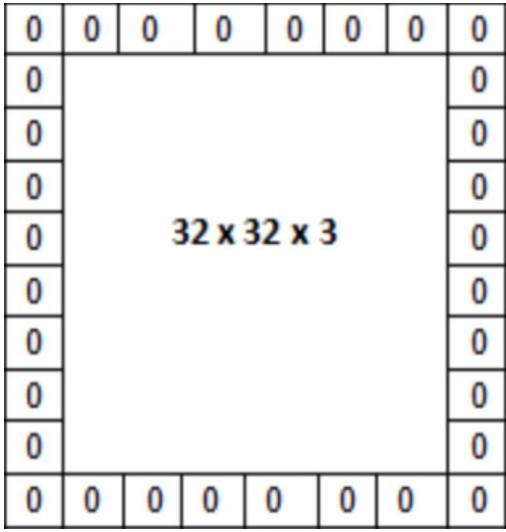
<hyperparameter>

1> valid (default)

⇒ padding을 주지 않는다.

2> same

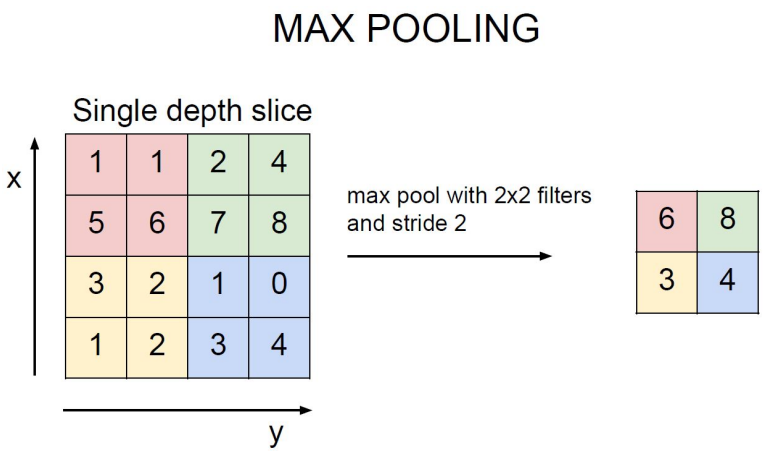
⇒ padding으로 output size와 input size가 같게끔 만든다. (depth를 제외하고)



4. CNN의 다른 옵션 2) pooling - maxpooling

1> 정의: 설정한 pool_size 안에서
가장 큰 값으로 downsampling

2> downsampling 해야 deeper layer에서
더 많은 영역의 정보를 담을 수 있습니다.
(쉽게 생각해서 중요한 것만 요약하고
나머지는 버린다고 생각하면 됩니다.)

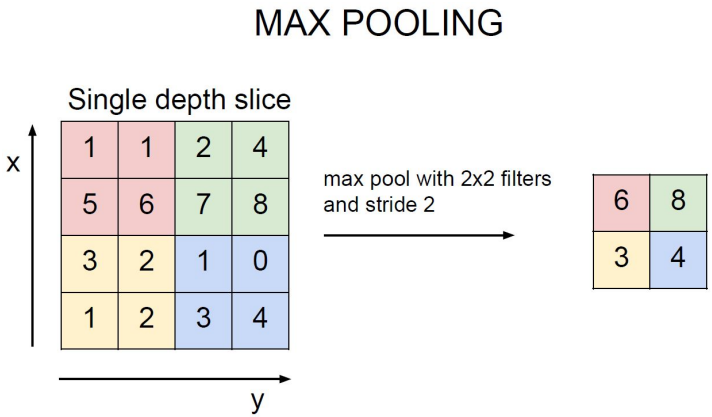


3> MaxPooling layer는 parameter가 있는 layer는 아닙니다.

4> 2, 3번의 이유로 overfitting 방지효과가 있습니다.

4. CNN의 다른 옵션 2) pooling - maxpooling

<hyperparameter>



1> pool_size = (2,2) (default)
(3,3)도 사용하기도 함
2> stride = None (default = pool_size)
(pooling이 3이면 stride는 2를 주로 사용)
3> padding = valid
(downsampling이 목적이기 때문에
겹치면서 <2번> 혹은 size를 유지하며<3번> maxpooling 하지는 않는다.)

일반적으로 convolution 1번 하면 maxpooling을 하지만
maxpooling은 필수로 해야하는 건 아닙니다.
(Conv2D 2회하고 Maxpooling2D 해도 됩니다.)

4. CNN의 다른 옵션 3) pooling - averagepooling

1> 정의: 설정한 pool_size 내부 값의 평균값으로 downsampling

2> 눈에 띄는 feature를 추출하는 게 좋기 때문에 maxpooling을 주로 사용합니다.

4. CNN의 다른 옵션 4) stride

1> 정의
convolution 연산을 몇 칸씩 이동하며 진행할 것인지

2> 이 또한 학습을 중간중간 빼먹으며 진행하기에 약간의 overfitting 방지효과가 있다.

3> pooling과 마찬가지로 downsampling을 목적으로 사용한다.

stride를 (2,2)로 convolution

0 0	0 0	1 0	0 0	0 1
0 0	1 1	0 0	1 1	0 0
1 0	0 0	0 0	0 0	1 0
0 0	1 1	0 0	1 1	0 0
0 1	0 0	1 0	0 0	0 0

바로 옆을 convolution 하는 게 아니라

4. CNN의 다른 옵션 4) stride

<hyperparameter>

1> strides = (1, 1) (default)

2> 최근 trendy한 논문들은 maxpooling 대신에 stride를 더 많이 쓰고 효과가 더 좋다고 해서 stride를 사용해보는 것도 좋습니다.

stride를 (2,2)로 convolution

0 0	0 0	1 0	0 0	0 1
0 0	1 1	0 0	1 1	0 0
1 0	0 0	0 0	0 0	1 0
0 0	1 1	0 0	1 1	0 0
0 1	0 0	1 0	0 0	0 0

바로 옆을 convolution 하는 게 아니라

4. CNN의 다른 옵션 5) output size 계산 공식

<Conv2D layer>

Summary. To summarize, the Conv Layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
  - Number of filters K,
  - their spatial extent F,
  - the stride S,
  - the amount of zero padding P.
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
  - $W_2 = (W_1 - F + 2P)/S + 1$
  - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. width and height are computed equally by symmetry)
  - $D_2 = K$
- With parameter sharing, it introduces $F \cdot F \cdot D_1$ weights per filter, for a total of $(F \cdot F \cdot D_1) \cdot K$ weights and K biases.
- In the output volume, the d-th depth slice (of size $W_2 \times H_2$) is the result of performing a valid convolution of the d-th filter over the input volume with a stride of S, and then offset by d-th bias.

(W-F+2P)/S가 정수로 나누어 떨어지지 않으면 버림 (가우스함수)을 한다고 생각하면 됩니다.

convolution이 불가능한 곳을 자동으로 **padding** 매겨서 하지는 않으니까요.

4. CNN의 다른 옵션 5) output size 계산 공식

<MaxPooling2D layer>

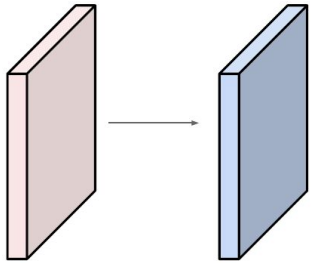
- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires three hyperparameters:
  - their spatial extent F,
  - the stride S,
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
  - $W_2 = (W_1 - F)/S + 1$
  - $H_2 = (H_1 - F)/S + 1$
  - $D_2 = D_1$
- Introduces zero parameters since it computes a fixed function of the input
- Note that it is not common to use zero-padding for Pooling layers

직접 계산해보는 시간
(계산 자체가 중요하지는 않지만 계산을 통해 내부구조를 이해할 수 있습니다.)

Examples time:

Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Output volume size: ?



Summary. To summarize, the Conv Layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
  - Number of filters K,
  - their spatial extent F,
  - the stride S,
  - the amount of zero padding P.
- Produces a volume of size $W_2 \times H_2 \times D_2$ v
  - $W_2 = (W_1 - F + 2P)/S + 1$
  - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. wi
  - $D_2 = K$

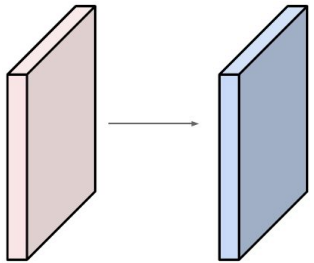
- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires three hyperparameters:
  - their spatial extent F,
  - the stride S,
- Produces a volume of size $W_2 \times H_2 \times D_2$
  - $W_2 = (W_1 - F)/S + 1$
  - $H_2 = (H_1 - F)/S + 1$
  - $D_2 = D_1$

직접 계산해보는 시간
(계산 자체가 중요하지는 않지만 계산을 통해 내부구조를 이해할 수 있습니다.)

Examples time:

Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Output volume size:
(32+2*2-5)/1+1 = 32 spatially, so
32x32x10



Summary. To summarize, the Conv Layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
  - Number of filters K,
  - their spatial extent F,
  - the stride S,
  - the amount of zero padding P.
- Produces a volume of size $W_2 \times H_2 \times D_2$ v
  - $W_2 = (W_1 - F + 2P)/S + 1$
  - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. wi
  - $D_2 = K$

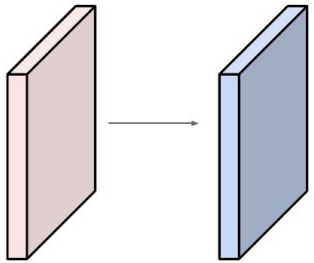
- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires three hyperparameters:
  - their spatial extent F,
  - the stride S,
- Produces a volume of size $W_2 \times H_2 \times D_2$
  - $W_2 = (W_1 - F)/S + 1$
  - $H_2 = (H_1 - F)/S + 1$
  - $D_2 = D_1$

직접 계산해보는 시간
(계산 자체가 중요하지는 않지만 계산을 통해 내부구조를 이해할 수 있습니다.)

Examples time:

Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Number of parameters in this layer?



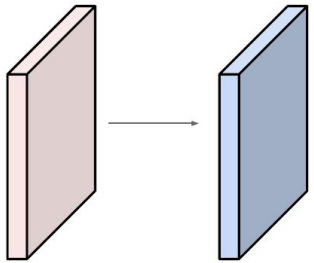
Summary. To summarize, the Conv Layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
  - Number of filters K,
  - their spatial extent F,
  - the stride S,
  - the amount of zero padding P.
- Produces a volume of size $W_2 \times H_2 \times D_2$ v
  - $W_2 = (W_1 - F + 2P)/S + 1$
  - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. wi
  - $D_2 = K$

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires three hyperparameters:
  - their spatial extent F,
  - the stride S,
- Produces a volume of size $W_2 \times H_2 \times D_2$
  - $W_2 = (W_1 - F)/S + 1$
  - $H_2 = (H_1 - F)/S + 1$
  - $D_2 = D_1$

직접 계산해보는 시간
(계산 자체가 중요하지는 않지만 계산을 통해 내부구조를 이해할 수 있습니다.)

Examples time:



Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Number of parameters in this layer?
each filter has 5*5*3 + 1 = 76 params (+1 for bias)
=> 76*10 = **760**

Summary. To summarize, the Conv Layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
  - Number of filters K,
  - their spatial extent F,
  - the stride S,
  - the amount of zero padding P.
- Produces a volume of size $W_2 \times H_2 \times D_2$ v
  - $W_2 = (W_1 - F + 2P)/S + 1$
  - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. wi
  - $D_2 = K$

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires three hyperparameters:
  - their spatial extent F,
  - the stride S,
- Produces a volume of size $W_2 \times H_2 \times D_2$
  - $W_2 = (W_1 - F)/S + 1$
  - $H_2 = (H_1 - F)/S + 1$
  - $D_2 = D_1$

직접 계산해보는 시간
(계산 자체가 중요하지는 않지만 계산을 통해 내부구조를 이해할 수 있습니다.)

```python
model = Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 3)))
model.add(MaxPooling2D((2, 2)))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D((2, 2)))
model.add(Conv2D(64, (3, 3), activation='relu'))
```

```python
model.summary()
```

```
Model: "sequential_2"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_1 (Conv2D)            [redacted]
_________________________________________________________________
max_pooling2d (MaxPooling2   [redacted]
_________________________________________________________________
conv2d_2 (Conv2D)            [redacted]
_________________________________________________________________
max_pooling2d_1 (MaxPoolin   [redacted]
_________________________________________________________________
conv2d_3 (Conv2D)            [redacted]
=================================================================
Total params: 56,320
Trainable params: 56,320
Non-trainable params: 0
_________________________________________________________________
```

Summary. To summarize, the Conv Layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
  - Number of filters K,
  - their spatial extent F,
  - the stride S,
  - the amount of zero padding P.
- Produces a volume of size $W_2 \times H_2 \times D_2$ v
  - $W_2 = (W_1 - F + 2P)/S + 1$
  - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. wi
  - $D_2 = K$

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires three hyperparameters:
  - their spatial extent F,
  - the stride S,
- Produces a volume of size $W_2 \times H_2 \times D_2$
  - $W_2 = (W_1 - F)/S + 1$
  - $H_2 = (H_1 - F)/S + 1$
  - $D_2 = D_1$

직접 계산해보는 시간
(계산 자체가 중요하지는 않지만 계산을 통해 내부구조를 이해할 수 있습니다.)

```python
model = Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 3)))
model.add(MaxPooling2D((2, 2)))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D((2, 2)))
model.add(Conv2D(64, (3, 3), activation='relu'))
```

```python
model.summary()
```

```
Model: "sequential_2"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_1 (Conv2D)            (None, 26, 26, 32)        896
_________________________________________________________________
max_pooling2d (MaxPooling2D) (None, 13, 13, 32)        0
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 11, 11, 64)        18496
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 5, 5, 64)          0
_________________________________________________________________
conv2d_3 (Conv2D)            (None, 3, 3, 64)          36928
=================================================================
Total params: 56,320
Trainable params: 56,320
Non-trainable params: 0
_________________________________________________________________
```

Summary. To summarize, the Conv Layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
  - Number of filters K,
  - their spatial extent F,
  - the stride S,
  - the amount of zero padding P.
- Produces a volume of size $W_2 \times H_2 \times D_2$ v
  - $W_2 = (W_1 - F + 2P)/S + 1$
  - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. wi
  - $D_2 = K$

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires three hyperparameters:
  - their spatial extent F,
  - the stride S,
- Produces a volume of size $W_2 \times H_2 \times D_2$
  - $W_2 = (W_1 - F)/S + 1$
  - $H_2 = (H_1 - F)/S + 1$
  - $D_2 = D_1$

5. Fully-Connected Layer 1) Dense layer의 필요성

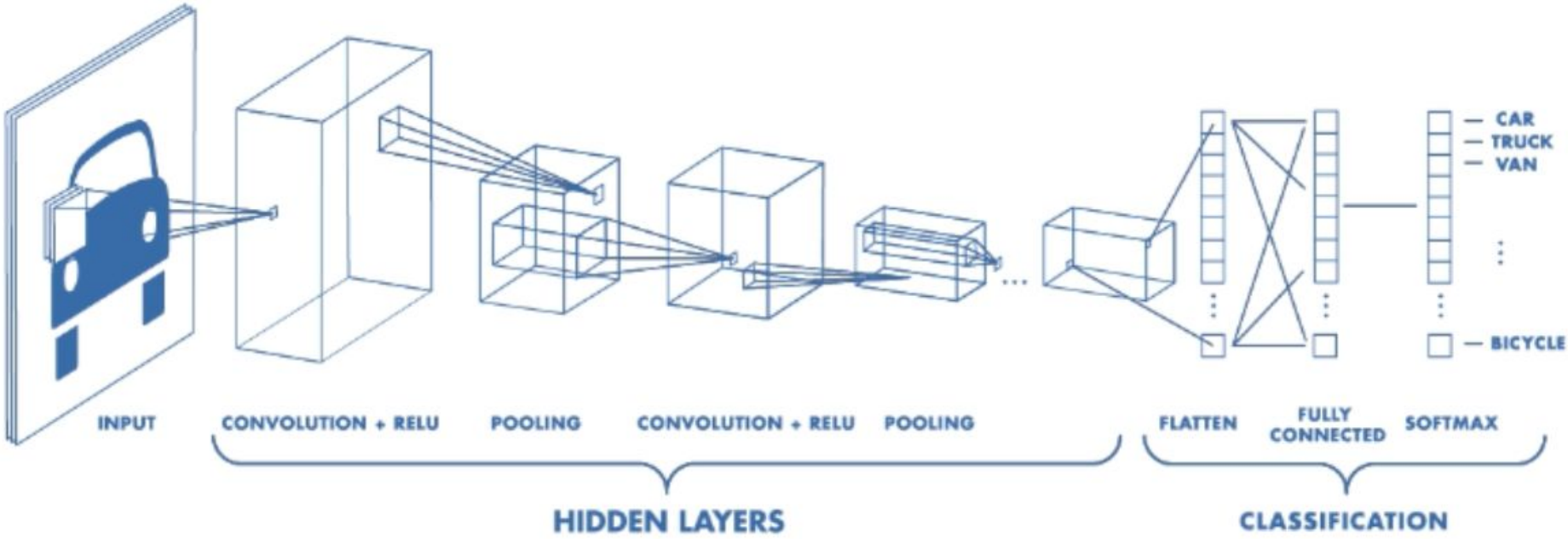
1> Convolutional layer 자체는 feature를 효율적으로 추출해줍니다.

- **Local-Connected: 인접 정보 추출**
- **Convolution: 인접 정보를 압축**
- **Deep: 인접 정보를 계층적으로 학습**

2> 하지만 Convolutional layer 자체가 classification 하지는 않습니다.
(Dense가 classification이 가능 / output이 scalar or vector로 출력되어서)

=> Convolutional layer 거친 뒤
=> Flatten layer 거쳐서
=> Dense layer를 거쳐야 합니다.

5. Fully-Connected Layer 2) 모델



RNN도 마찬가지로 마지막에 **Flatten**을 해서
Dense Layer에 값을 전달해주어야 합니다.

그림 잘 그린 사이트: http://colah.github.io/posts/2014-07-Understanding-Convolutions/